

Floyd Warshall Algorithm

모든 노드 쌍 (i, j) 사이의 최단거리를 구하는 DP 알고리즘.

All-Pairs Shortest Path (APSP)

※ Dijkstra는 한 시작점에서 모든 노드까지

※ 기본경로 $i \longrightarrow j \Rightarrow \text{dist}[i][j]$

새로운 경로 $i \longrightarrow k \longrightarrow j \Rightarrow \text{dist}[i][k] + \text{dist}[k][j]$

$\therefore \text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

※ 3중 for loop

for k in range(n):

for i in range(n):

for j in range(n):

...

Intermediate node

Leetcode : network delay time

```
dist = [[float('inf')] * n for _ in range(n)]
```

```
for u, v, w in edges:
```

```
    dist[u-1][v-1] = w
```

```
for i in range(n):
```

```
    dist[i][i] = 0
```

```
for mid in range(n):
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            dist[i][j] = min(dist[i][j],  
                              dist[i][mid] +  
                              dist[mid][j])
```

```
res = max(dist[k-1])
```

```
return res if res < float('inf') else -1
```

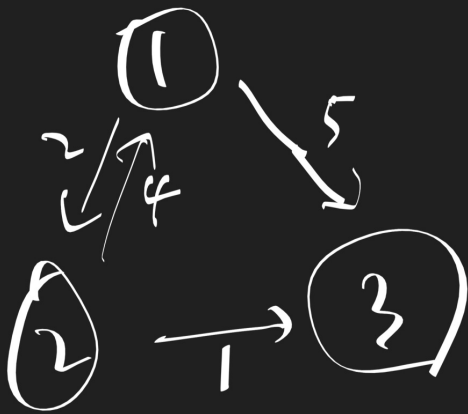
Floyd Warshall Algorithm

★ 거리가 있는 것 제거

- Dijkstra Algorithm은 최소 edge $\rightarrow$ 그리 갱신
heapq ... minheap... 1D Array & Dist.
- 모든 쌍의 최단 경로 (All Pairs shortest Path)
- Floyd Warshall $\rightarrow$ 2D array

0	2	5
∞	0	1
∞	4	0

$n \times n$



- 거리가 있는 것 $\rightarrow$ 0이 되는 것들을 생각해서 계산

ex) 1 $\rightarrow$ 2 $\rightarrow$ 1 갱신 가능
 $2 \rightarrow 3 = 1$ $\checkmark$ Floyd warshall
 $\infty \rightarrow 1 + 1 \rightarrow 3$
 $\infty + 5$

recurrence relation ($\mathcal{O}(n^3)$)

$$d[i][j] = \min(d[i][k], d[k][j] + d[k][i])$$

$$n = 4, \text{ edges} = \left[\begin{pmatrix} u & v & w \end{pmatrix}, \dots \right]$$

$$\text{dist} = \left[\left[\text{float('inf')} \right] * n \text{ for } _ \text{ in range}(n) \right]$$

for i in range(n):

$$\text{dist}[i][i] = 0$$

for u, v, w in edges:

$$\begin{aligned} \text{dist}[u][v] &= \min(\text{dist}[u][v], w) \\ \# \text{dist}[v][u] &= \min(\text{dist}[v][u], w) \\ &\text{if undirected} \end{aligned}$$

for k in range(n):

for i in range(n):

for j in range(n):

$$\text{if dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$$

$$\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$$

1) 선택 노드 $\rightarrow$ 최소 노드

$\Rightarrow$ 최소 노드 선택 노드 선택?

$\Rightarrow$ Prim $O(E \log V)$

All $\rightarrow$ Best $\rightarrow$ Cost $\rightarrow$ Dist

$\Rightarrow$ 최소 노드 선택? 노드 선택?

$\Rightarrow$ Floyd warshall

$\Rightarrow O(N^3)$

$\Rightarrow N \leq 500$